Major Project: Status Report

SRM Institute of Science and Technology

24 October 2026

1. Team Members

No.	Name
1	Aakash G S
2	Hariharan
3	Jaswanth

2. Proposed Project Title

MCP-Lock: Lockfiles and Pre-Launch Gating for Coding-Agent MCP Servers

3. Problem Statement

AI coding agents (for example Claude Code and Cursor) get their tools from *Model Context Protocol (MCP) servers*, small programs that agents usually download and start from public package registries with commands such as `npx -y package` or `uvx package`. These commands are *unpinned*: every launch silently fetches the newest version. Two things can therefore change between one session and the next without the user noticing:

- the **code** that runs on the developer's machine, which can carry malware if a maintainer account is hijacked or a malicious update is published; and
- the **tool descriptions** that the AI model reads as instructions, which an attacker can rewrite after the user has approved the server (a "rug pull" that injects hidden instructions into the model).

Existing pre-install checks look only at package names, sources and age, and registry provenance records who built a package, not what tools it exposes. Our census found that **36.5%** of 2,000 sampled MCP packages changed their tools within 90 days, and that **97.0%** of those changes carried no warning sign in registry metadata.

Our project solves this with MCP-Lock: a lockfile that records the exact reviewed state of every MCP server (version, archive digest, build provenance, publisher, install scripts and a hash of its tool definitions), and a gate that checks every launch against it *before* any package code runs, blocking or flagging anything that changed.

4. Current Status

Stage	Status
Discussion stage	✓ Completed
Topic finalization	✓ Completed
Literature survey	✓ Completed (30 cited works, citations verified)
Design	✓ Completed (lockfile specification and gate design)
Implementation	✓ Completed (working tool, 103 automated tests passing)
Evaluation	✓ Completed (pre-registered evaluation and adaptive-attacker study)
Research paper	▶ In progress (6-page IEEE-format draft, revised after an internal review)

Overall stage: implementation and evaluation are complete; the team is now finalizing the research paper.

5. Work Completed So Far

Phase	Outcome
Literature survey and baseline analysis	Surveyed MCP security, agent supply-chain and package supply-chain work. Re-implemented the closest existing defence (the Bagmar and Saraf pre-install hook, arXiv:2607.15143) as our baseline.
Census of MCP packages	Collected metadata for all 15,272 eligible MCP packages on npm and PyPI (228,044 releases), and statically extracted tool definitions from a stratified sample of 2,000. Validated the extractor against live servers in an isolated container (precision 92.2%, recall 78.0%).
Design and implementation	Built the `mcp-lock.json` lockfile format with a written specification and JSON schema, the `mcplock` command-line tool, and a Claude Code hook that checks every install or launch command.
Pre-registered evaluation	Registered hypotheses and analysis plan on OSF before running (DOI 10.17605/OSF.IO/2AMJ3). On 490 real attacks MCP-Lock detected **39.8%** against **19.6%** for the baseline (99 more attacks; exact McNemar $p = 3.2 \times 10^{-30}$). It caught all 40 of 40 synthetic rug pulls, which the baseline missed entirely.
Adaptive-attacker study	Built 12 attacks by an attacker who knows every check. The basic gate blocks 1 and flags 4 of them; five optional hardening switches close all but 2, and the cost of each was measured on 8,180 honest upgrades. Ran every test attack server in a sealed container to confirm the runtime results (11 of 11 agree).
Safer-alternative suggestions	When a command is blocked, the gate now proposes a verified safe replacement (for example the last clean version), shown only if it passes the gate. Correct for 114 of 114 hijacked packages that received a suggestion.
Research paper	Wrote a 6-page IEEE-format paper in which every number is generated from the result files. Ran a simulated IEEE-style peer review and fixed 9 of its 10 points: added comparisons with simpler alternatives, prior work on tool pinning, and robustness checks.

Code, data and results are public at `https://github.com/ftaakash/MCP-Lock`; the pre-registration is at `https://osf.io/2amj3`.

6. Next Tasks Planned

1. **Guide review of the paper draft:** incorporate feedback from our project guide.
2. **Select the target venue:** choose an IEEE conference or workshop, then adjust the page limit, anonymisation and formatting to its rules.
3. **Expand the methods section:** describe the six documented deviations from the pre-registration in full, the last open point of the internal review.
4. **Final checks before submission:** repeat the related-work search, since the field changes monthly; proofread; and verify every reference.
5. **Submit the paper,** and optionally post a preprint on arXiv.

7. Difficulties and Support Required

- **Choice of venue and timeline:** we would value the guide's advice on a suitable IEEE conference or workshop and its submission deadline.
- **Independent evaluation:** the adaptive attacks were designed by our own team; feedback from the guide or another faculty member on further attacks would strengthen the evaluation.
- **Limited real-world data:** no public dataset of real tool-description rug pulls exists, so that part of the evaluation relies on synthetic and adaptive cases. We have stated this openly in the paper.
- **Prior work on tool pinning:** recent open-source tools already pin MCP tool definitions, so we have repositioned the contribution around pre-launch checking and the measured evaluation. We would welcome the guide's view on this framing.